

回溯算法

《算法设计与分析》

信息科学与技术学院 曾华琳



01



算 法 思 想



回溯(Backtracking)

- 介绍

假如你要在许多不同的选择中做一系列的决策，那么

- 你没有足够的信息来帮助做出选择
- 每个决策都将产生一些新的选择
- 一些决策的序列(可能不只一个)也许就是问题的解

回溯算法系统的尝试搜索各种决策序列，直到找到一个令你满意的决策序列



术语

一棵由节点构成的树。有3类节点被用到

● 根节点(开始节点)

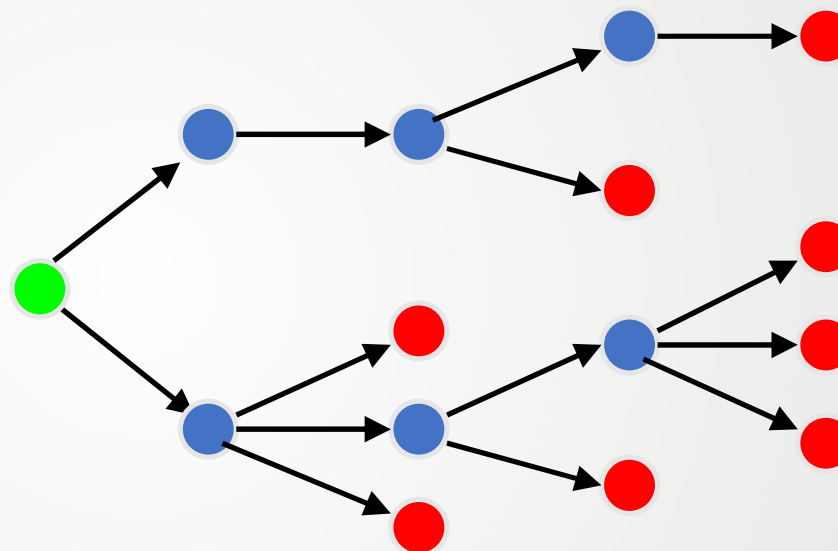
● 内部节点

● 叶子节点

● 活动节点

● 扩展节点

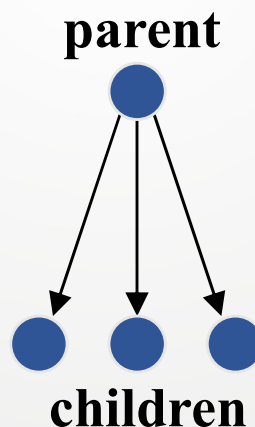
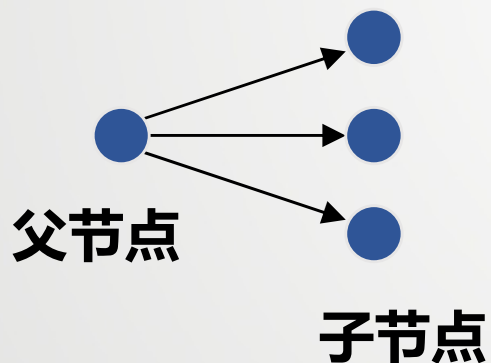
● 死节点



回溯算法可以看作是为了找
某个特定的叶子节点而进行
系统搜索的一种方法

>>> 术语

- 在一棵树中,非叶子节点是其他一个或多个节点的父节点(它的子节点)
- 树中的每个节点(除了根节点) 都有一个父节点



通常我们都从下画树。
根节点在顶部



回溯(Backtracking)

回溯法是一个既带有系统性又带有跳跃性的搜索算法；

它在包含问题的所有解的解空间树中，按照深度优先的策略，从根结点出发搜索解空间树。—— **系统性**

算法搜索至解空间树的任一结点时，判断该结点为根的子树是否包含问题的解，如果肯定不包含，则跳过以该结点为根的子树的搜索，逐层向其祖先结点回溯。否则，进入该子树，继续深度优先的策略进行搜索。—— **跳跃性**

这种以深度优先的方式系统地搜索问题的解得算法称为回溯法，它适用于解一些组合数较大的问题。



回溯(Backtracking)

- 回溯的思想

回溯是穷举方法的一个改进，它在所有可行的选择中，系统地搜索问题的解。它假定解可以由向量形式 $(x_1, x_2, \dots, x_n)$ 来表示，其中 x_i 的值表示在第 i 次选择所作的决策值，并以深度优先的方式遍历向量空间，逐步确定 x_i 的值，直到解被找到。

- 回溯法列举出解空间中所有可能的情形来确保解的正确性。



回溯(Backtracking)

- 假设我们已经确定部分解的集合 $(x_1, x_2, \dots, x_i)$ ，在此基础上通过增加解元素 x_{i+1} 来扩展解，确定 x_{i+1} 的值后，我们要测试 $(x_1, x_2, \dots, x_{i+1})$ 是否仍是可行解。

➤➤➤ 回溯(Backtracking)

回溯算法的基本步骤如下

1. 定义问题的解空间。这个空间必须包含一个问题的最优解。

解的空间

如果 S_i 是 x_i 的范围, 那么 $S_1 \times S_2 \times \dots \times S_n$ 是问题的解空间



回溯(Backtracking)

通常这个解空间是非常巨大的，所以搜索一个目标解的代价是很难想象的。为了使回溯算法更有效率，我们必须缩小搜索空间。

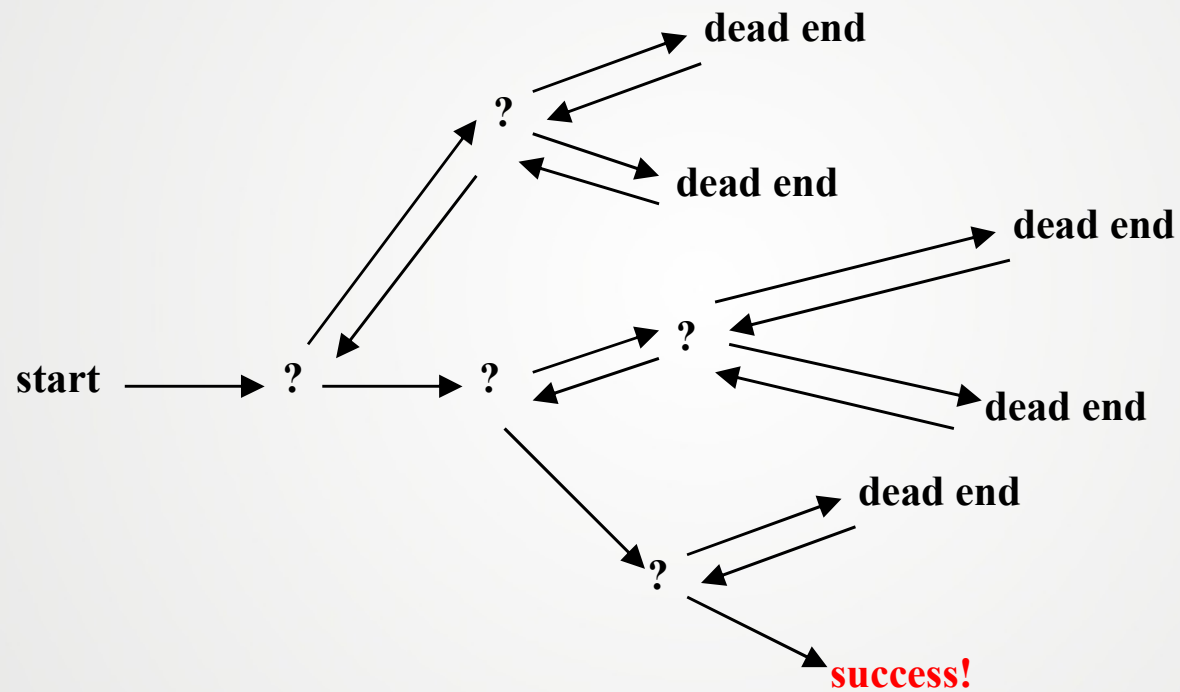
2. 组织好解空间以便使搜索更容易

典型的组织方式是利用图或树的结构。

3. 以深度优先方式搜索解空间，利用剪枝函数来避免搜索进入不可能得到解的子空间。



解空间树(Solution Space Tree)



➤➤➤ 生成问题状态的基本方法

- **扩展结点:**一个正在产生儿子的结点称为扩展结点
- **活结点:**一个自身已生成但其儿子还没有全部生成的节点称做活结点
- **死结点:**一个所有儿子已经产生的结点称做死结点
- **深度优先的问题状态生成法:** 如果对一个扩展结点R, 一旦产生了它的一个儿子C, 就把C当做新的扩展结点。在完成对子树C (以C为根的子树) 的穷尽搜索之后, 将R重新变成扩展结点, 继续生成R的下一个儿子 (如果存在)
- **宽度优先的问题状态生成法:** 在一个扩展结点变成死结点之前, 它一直是扩展结点
- **回溯法:** 为了避免生成那些不可能产生最佳解的问题状态, 要不断地利用限界函数(bounding function)来处死那些实际上不可能产生所需解的活结点, 以减少问题的计算量。具有有限界函数的深度优先生成法称为回溯法



解空间的组织

- 任何时候，在构造问题解的时候，都包含做出一系列的决策，如果我们将做出的决策画出来，这个图就象一棵树。建立一个树型结构，使得树叶对应解空间的一个解，而内部节点的一个分支，对应一个决策，这样，便可以将解空间组织为一棵解空间树。
- 回溯法可以很容易地被用来搜索一个集合的所有子集合或是排列。

解空间的组织

- 当我们要解决的问题是要求一个使问题最优的 n 个元素的子集，问题的解空间常可以组织成一棵子集树

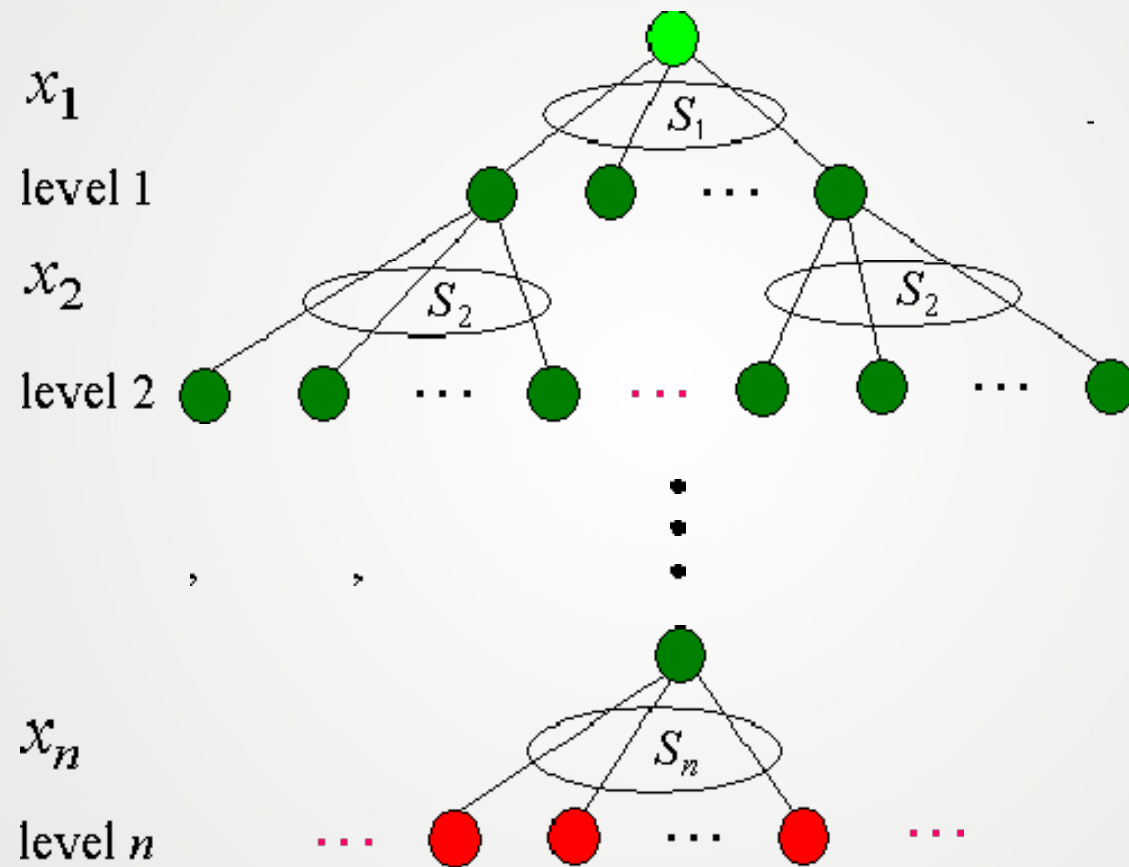
- 构造所有的子集

n 个元素的集合有多少个子集？

如果每个 S_i 的大小是 k ，对每个 $x_i \in S_i$ ，共有 k^n 个子集

- 子集树

子集树(Subtree)



子集树的程序模版(Program Template for Subtree)

Backtrack(i)

```
1  if  $i > n$  then Update( $x$ )  
2  else  
3      for each  $a \in S_i$  do  
4           $x_i \leftarrow a$   
5          if  $C(i)$  and  $B(i)$  then  
6              Backtrack( $i+1$ )
```

- 当问题是求 n 个元素一个排列以使问题最优化时，解空间常可以组织成一棵排列树。

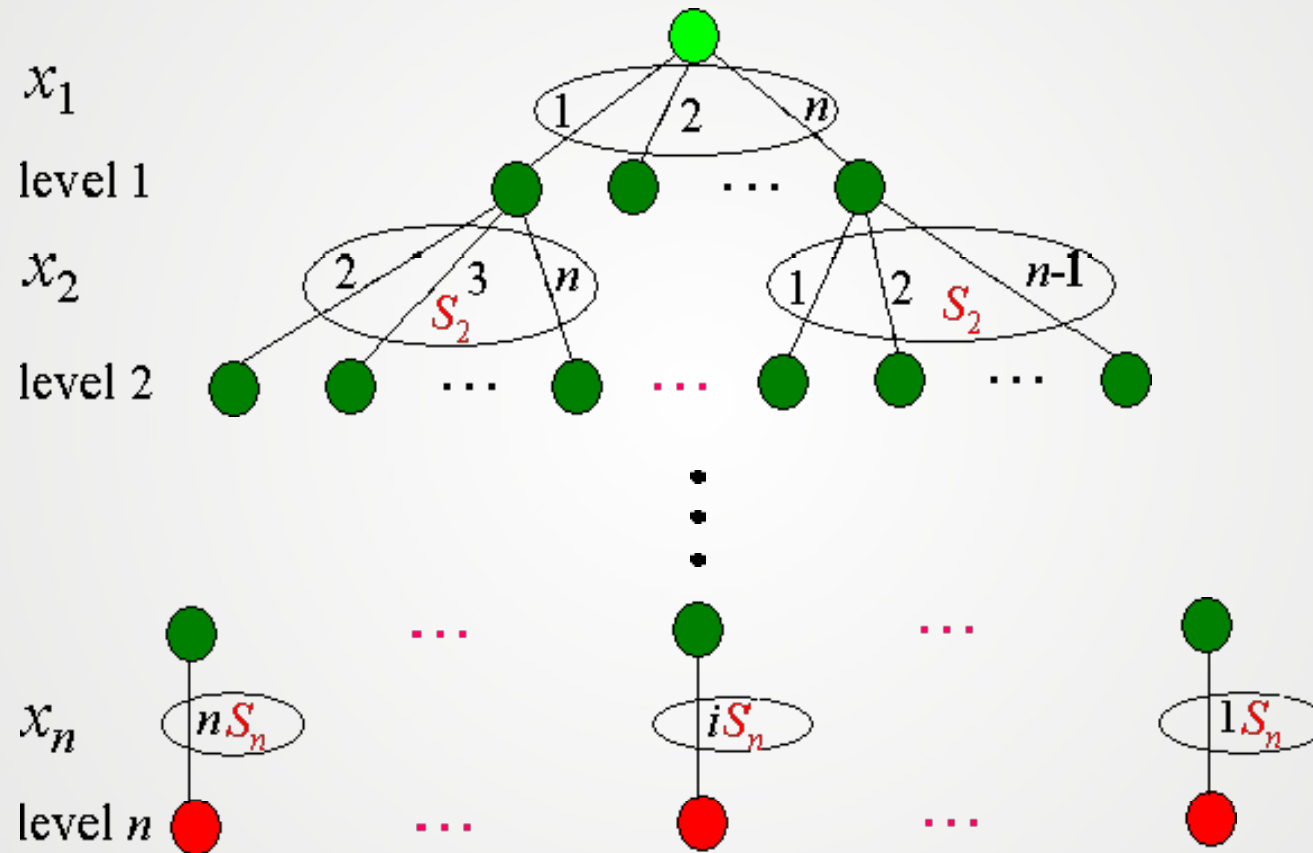
- 构造所有的排列

n 个元素的集合有多少种的排列？

$n!$

- 排列树

>>> 排列树(Permutation tree)





Program Template for Permutation tree

Backtrack(i)

```
1 if  $i > n$  then Update( $x$ )  
2 else  
3   for  $j \leftarrow i$  to  $n$  do  
4      $x_i \leftrightarrow x_j$   
5     if  $C(i)$  and  $B(i)$  then  
6       Backtrack( $i+1$ )  
7      $x_i \leftrightarrow x_j$ 
```

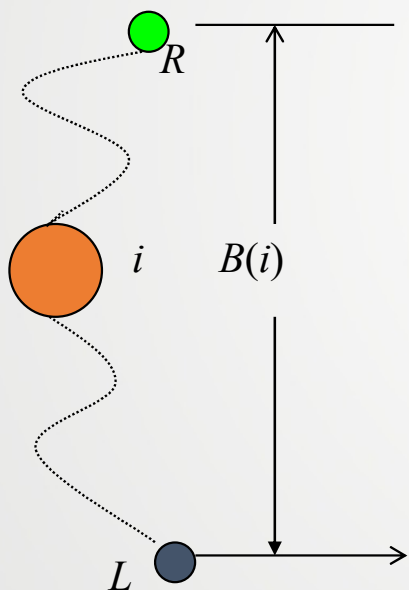


剪枝函数

- 剪枝函数一般有两种：**约束函数**和**限界函数**
- 利用剪枝函数，剪除无效的分支,集中搜索最有用的分支.
- 为了改进搜索，应用回溯法至少需要注意以下四点：
 - ① 怎样选择约束函数。
 - ② 怎样计算上界 (对最大化问题)
 - ③ 怎样计算下界 (对最小化问题)
 - ④ 怎样利用约束函数和限界函数来进行剪枝.

>>> 剪枝函数

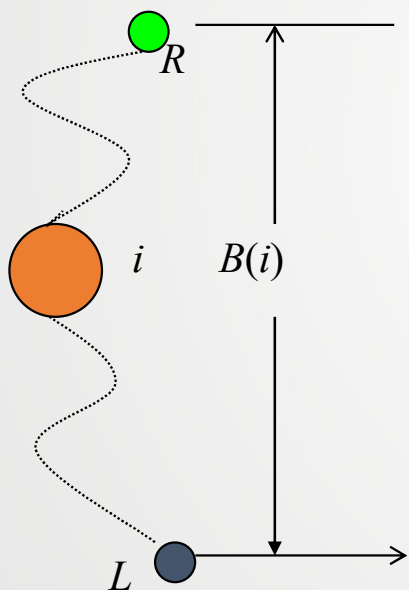
- 通常，对极大值问题，我们对扩展节点 i 计算上界 $B(i)$ 。如果目前保存的最大目标值不比 $B(i)$ 大，那么进行剪枝，否则继续。



从根节点 R 通过 i 到叶子 L 的任一决策序列的目标函数值不能比 $B(i)$ 大，所以如果 $B(i) \leq \text{bestc}$ ，那么它表明搜索扩展节点 i 不会成功，因此对扩展节点 i 进行剪枝。

>>> 剪枝函数

- 通常，对极小值问题，我们对扩展节点 i 计算上界 $B(i)$ 。如果目前保存的最大目标值不比 $B(i)$ 小，那么进行剪枝，否则继续。



从根节点 R 到叶子 L 通过 i 的任一决策序列的目标函数值不能比 $B(i)$ 小，所以如果 $B(i) \geq \text{bestc}$ ，那么它表明搜索扩展节点 i 不会成功，因此对扩展节点 i 进行剪枝。

A person's hand is visible on the left, holding a silver marker and drawing a circle on a whiteboard. The person is also holding a document with a colorful diagram consisting of four circles (green, blue, orange, red) arranged in a square. The background is a bright, modern office space with a large window. The text "谢谢大家" is written in large blue characters on the right side of the whiteboard.

谢谢大家